

Gestion des données et personnalisation de UI Avec Flutter



Année universitaire 2025-2026

Gestion des Données



Stockage de données

A thick, hand-drawn style orange line underlining the text "Stockage de données".

Stockage de Données Local

Le stockage de données local se réfère aux données stockées directement sur l'appareil de l'utilisateur, ne nécessitant pas de connexion réseau.

Méthodes courantes :

Shared Preferences

SQLite

Fichiers

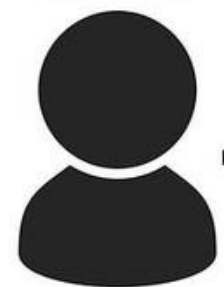
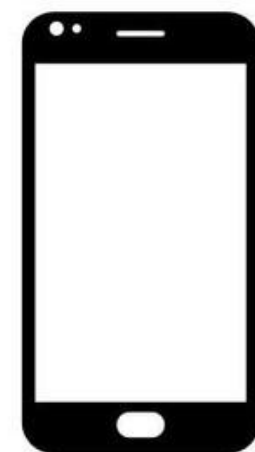


Shared Preferences

Utilisé pour stocker de petites quantités de données sous forme de paires clé-valeur.

Avantages :

- Rapide et facile à mettre en œuvre.
- Idéal pour stocker des préférences utilisateur simples.



{ name
age
location



Shared
Preferences



SQLite

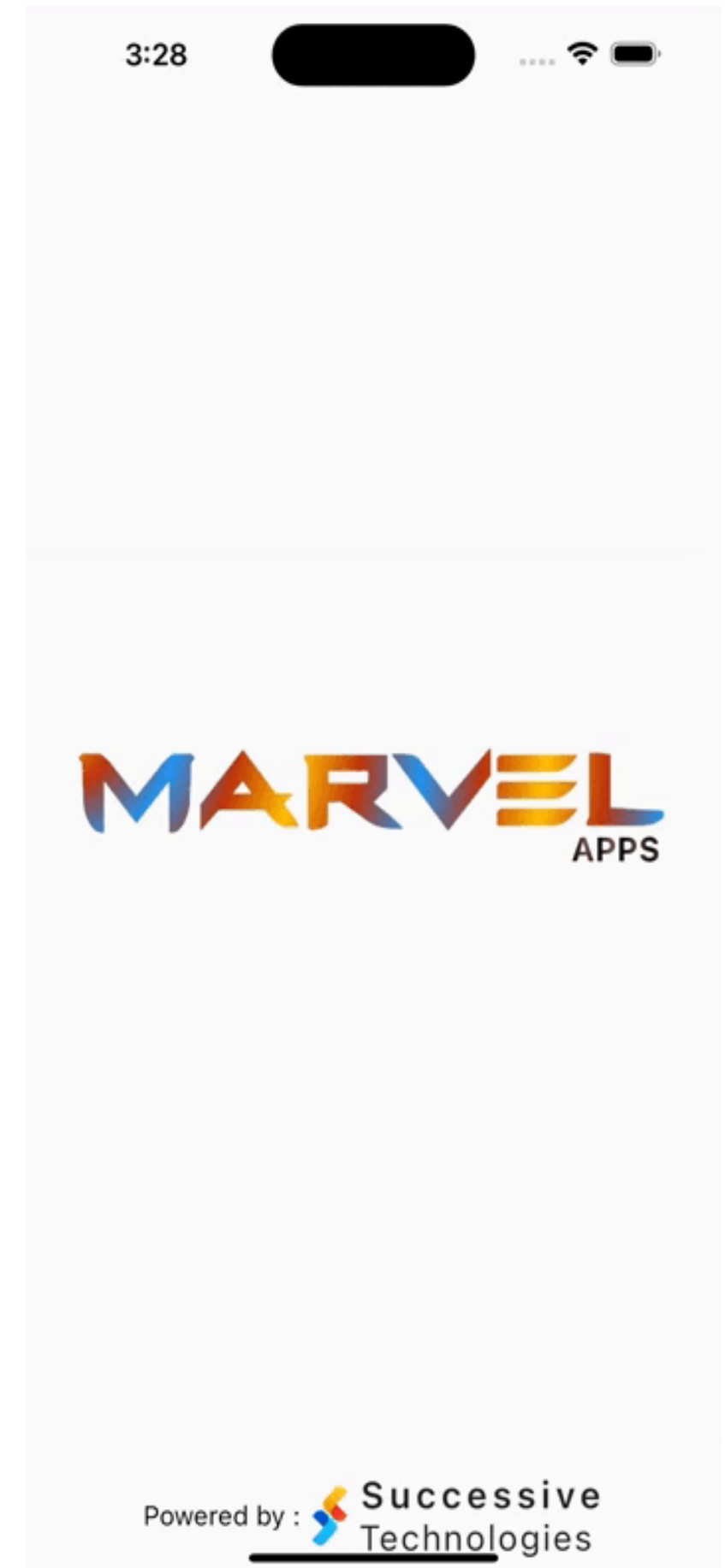
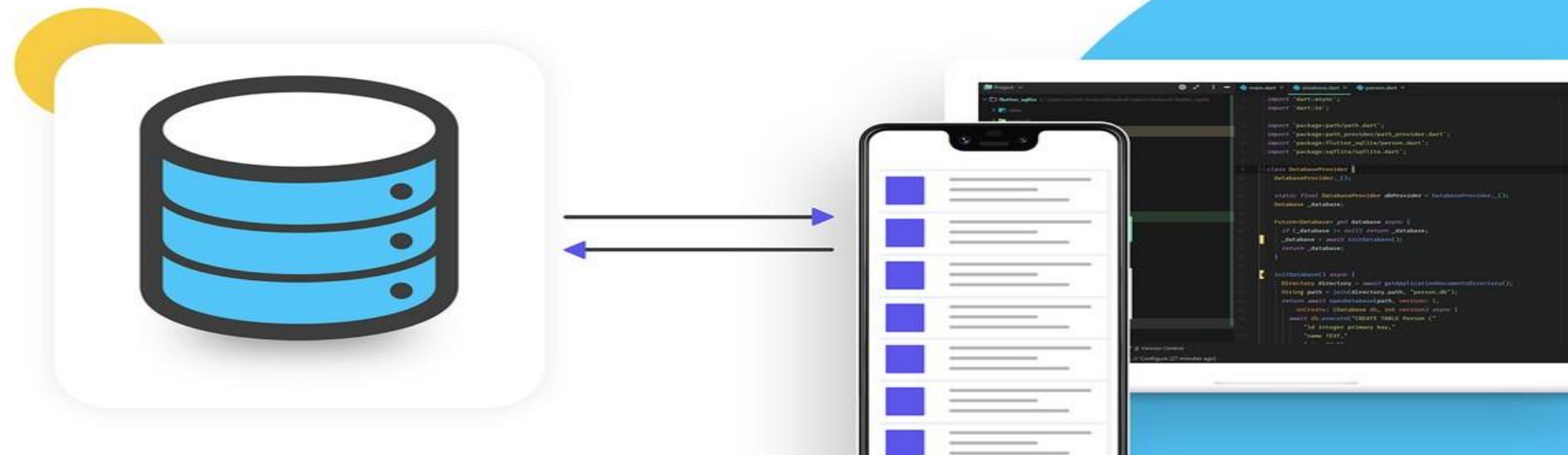
Une base de données relationnelle utilisée pour stocker des données structurées.

Avantages :

Gestion efficace des données structurées.

Prise en charge de requêtes complexes et de transactions.

■ SQLITE IN FLUTTER



Activité 1

Interactions avec la base de données

CRUD : sqflite

Les applications Flutter peuvent utiliser les bases de données SQLite via le plugin **sqflite** disponible sur pub.dev. Cette recette montre les bases de l'utilisation **sqflite** pour insérer, lire, mettre à jour et supprimer des données.

1. Ajouter les dépendances

dependencies:

```
flutter:  
  sdk: flutter  
sqflite:  
path:
```

Importer les packages dans le fichier dans lequel vous allez travailler.

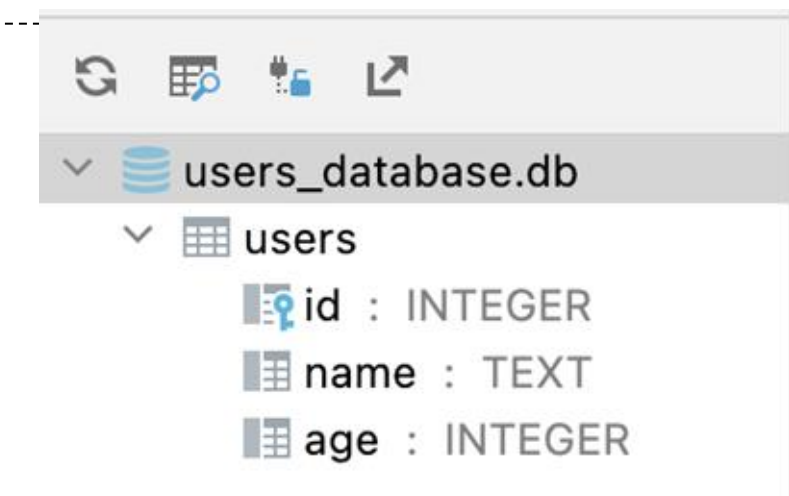
```
import 'dart:async';  
import 'package:flutter/widgets.dart';  
import 'package:path/path.dart';  
import 'package:sqflite/sqflite.dart';
```

2. Définir le modèle de données User

```
class User {  
  final int id;  
  final String name;  
  final int age;  
  
  const User({  
    required this.id,  
    required this.name,  
    required this.age,  
  });  
}
```

3. Ouvrez la base de données

```
WidgetsFlutterBinding.ensureInitialized();  
  
final database = openDatabase(  
  join(await getDatabasesPath(),  
    'users_database.db'),  
);
```



La création de la base de données

Remarque

- Pour utiliser le mot-clé **await**, le code doit être placé à l'intérieur d'une async fonction. Vous devez placer toutes les fonctions de table suivantes à l'intérieur de **void main() async {}**.

Activité 1

Interactions avec la base de données

CRUD : sqflite

4. Créez le tableau Users

```
final database = openDatabase(
  join(await getDatabasesPath(), 'users_database.db'),
  onCreate: (db, version) {
    return db.execute(
      'CREATE TABLE users(id INTEGER PRIMARY KEY,
name TEXT, age INTEGER)',
    );},
  version: 1,
);
```

users			
Live updates			
	id	name	age
	1	user	54

L'exécution de la méthode **insertUser()**

5. Insérez un User dans la base de données

- Convertissez l'User en un **Map**

```
Map<String, dynamic> toMap() {
  return {
    'id': id,
    'name': name,
    'age': age,
  };
}
```

```
@override
String toString() {
  return 'User{id: $id, name: $name, age:
$age}';
}
```

- Utilisez la méthode **insert()** pour stocker le **Map** dans la table Users.

```
Future<void> insertUser(User user) async {
  final db = await database;

  await db.insert(
    'users',
    user.toMap(),
    conflictAlgorithm: ConflictAlgorithm.replace,
  );
}
```

```
// Create a User and add it to the users table
var fido = const User(
  id: 0,
  name: 'Fido',
  age: 35,
);

await insertUser(fido);
```

Activité 1

Interactions avec la base de données

CRUD : sqflite

6. Récupérer la liste des Users

```
Future<List<User>> users() async {  
  final db = await database;  
  final List<Map<String, dynamic>> maps =  
    await db.query('users');  
  
  return List.generate(maps.length, (i) {  
  
    return User(  
      id: maps[i]['id'],  
      name: maps[i]['name'],  
      age: maps[i]['age'],  
    );  
  });  
}
```

```
// Now, use the method above to retrieve all the  
Users.  
print(await Users()); // Prints a list that include  
Fido.
```

7. Mettre à jour un User dans la base de données.

```
Future<void> updateUser(User user) async {  
  final db = await database;  
  
  await db.update(  
    'users',  
    user.toMap(),  
    where: 'id = ?',  
    whereArgs: [user.id],  
  );  
  updateUser(User (  
    id: 1,  
    name: "fido",  
    age: 37  
  ));  
}
```



Remarque

- Utilisez toujours `whereArgs` pour passer des arguments à une `where` instruction. Cela permet de se prémunir contre les attaques par injection SQL.
- N'utilisez pas d'interpolation de chaîne, telle que `where: "id = ${user.id}"!`

users			
Live updates			
	id	name	age
1	1	fido	37

L'exécution de la méthode **UpdateUser()**

Activité 1

Interactions avec la base de données

CRUD : sqflite

8. Supprimer un User de la base de données

```
Future<void> deleteUser(int id) async {  
  // Get a reference to the database.  
  final db = await database;  
  
  // Remove the User from the database.  
  await db.delete(  
    'users',  
  
    where: 'id = ?',  
  
    whereArgs: [id],  
  );  
}
```

La méthode qui permet de supprimer un **élément** de la base de données

☒ Live updates 

 id	name	age
--	------	-----

L'exécution de la méthode **deleteUser()**



Remarque

- Utilisez toujours un message d'alerte avant la suppression d'un élément pour demander la confirmation de l'utilisateur.

Fichiers

Utilisés pour stocker des données non structurées ou des fichiers volumineux directement sur le périphérique.

Avantages :

Flexibilité pour stocker une variété de données, y compris des fichiers multimédias.

Contrôle direct sur le stockage et la manipulation des fichiers.

Exemple d'utilisation :

Stockage de fichiers image, audio ou vidéo, ainsi que de tout type de données non structurées nécessaires à l'application.

Stockage de Données Distant

Le stockage de données distant concerne les données stockées sur des serveurs distants accessibles via une connexion réseau.

Méthodes courantes :

Firestore Realtime Database

Cloud Firestore

API REST





Firebase

Realtime Database

Firebase

- Base de données NoSQL en temps réel offerte par Google Firebase.

Realtime

- **Avantages** :
 - Synchronisation en temps réel des données entre les clients et le serveur. Facilité d'utilisation et intégration avec d'autres services Firebase.

Database

Cloud Firestore

Base de données NoSQL flexible et évolutive offerte par Google Firebase.

Avantages :

Structure de données hiérarchique et évolutivité horizontale.

Prise en charge de requêtes complexes et d'indexation automatique.



Cloud
Firestore

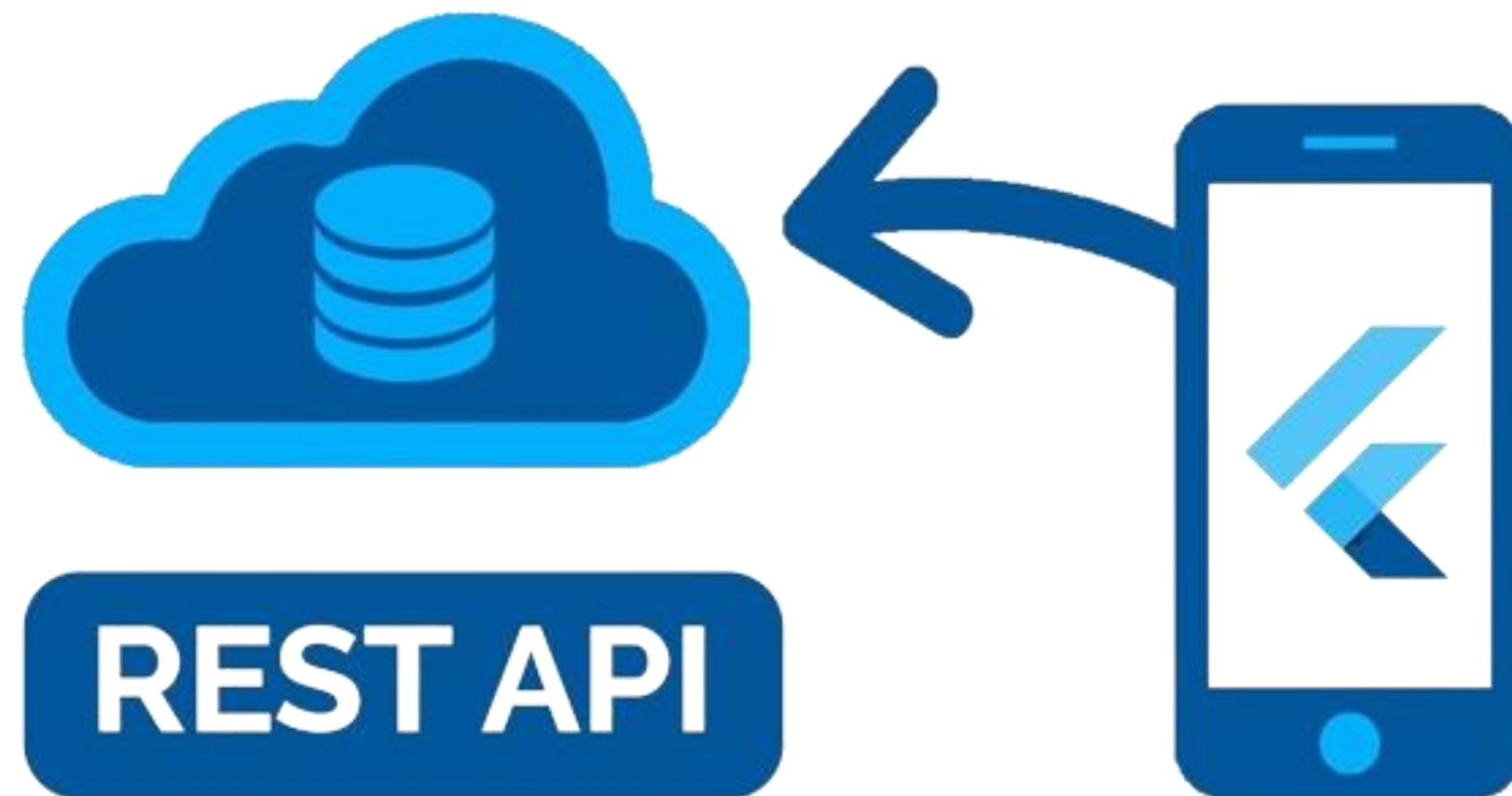
API REST

Accès aux données back-end à l'aide de requêtes HTTP conformes au style architectural REST.

Avantages :

Flexibilité pour accéder à différentes ressources à partir de divers serveurs.

Interopérabilité avec différentes plateformes et langages de programmation.



Gestion des Données dans l'État de l'Application



Gestion des Données dans l'État de l'Application

- La gestion efficace de l'état de l'application est cruciale pour garantir une expérience utilisateur fluide et réactive.

1. Provider

Bibliothèque permettant de gérer l'état global de l'application en fournissant une solution simple et légère.

2. BLoC (Business Logic Component)

Modèle d'architecture d'application réactif qui sépare la logique métier de l'interface utilisateur.

3. GetX

Bibliothèque offrant une gestion simplifiée de l'état et de la navigation dans les applications Flutter.

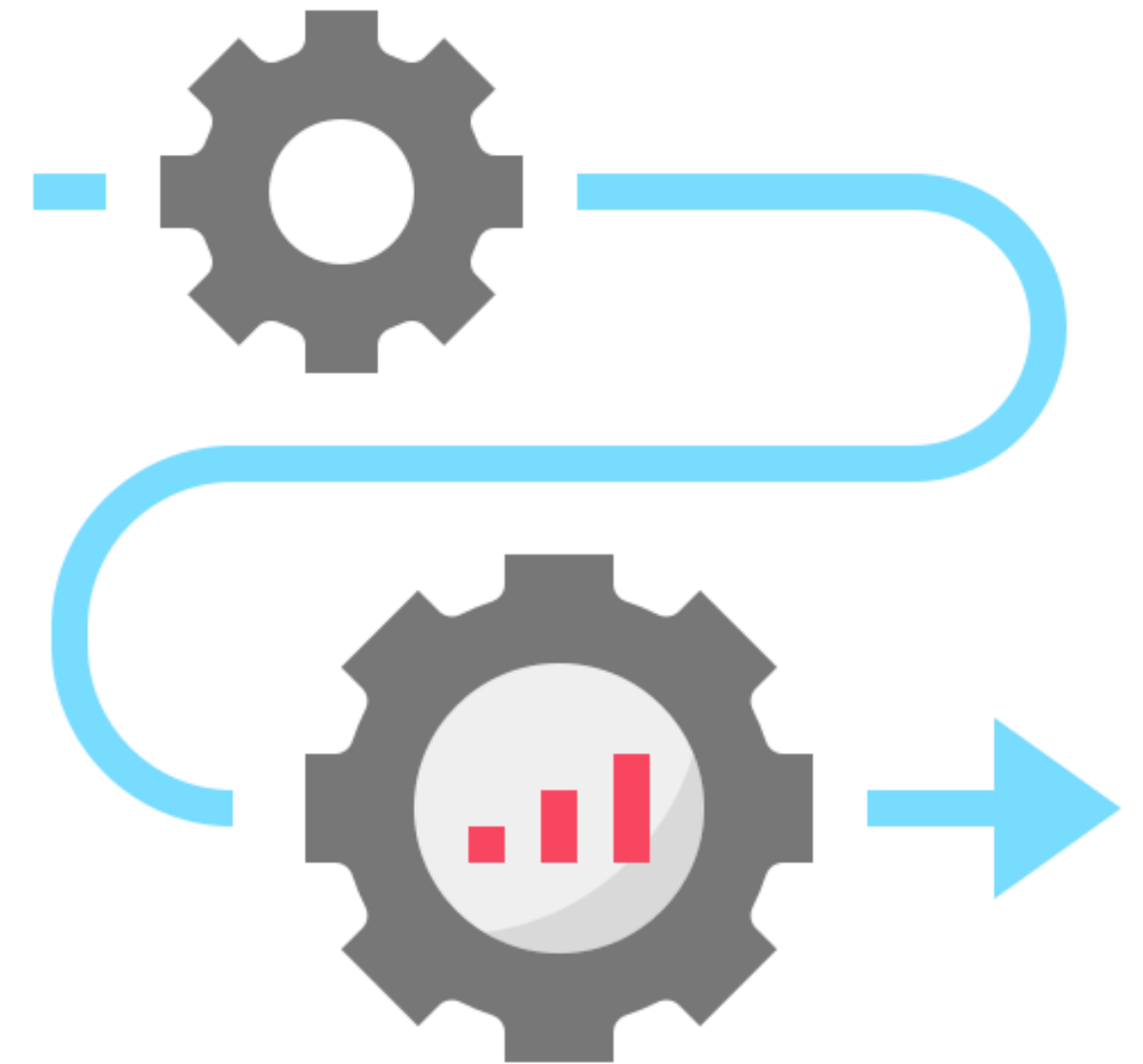
Meilleures Pratiques



Choisir l'Approche Adaptée

Sélectionner une méthode de stockage appropriée en fonction des besoins spécifiques de l'application

Prévoir les évolutions futures de l'application pour choisir une solution flexible et évolutive



Sécuriser les Données Sensibles

Mettre en place des mesures de sécurité robustes pour protéger les données sensibles contre les accès non autorisés

Renforcer la sécurité informatique par une gestion rigoureuse des accès et des audits.



Optimiser les Requêtes de Données

Minimiser les temps de chargement en optimisant les requêtes de données, en utilisant la mise en cache et la pagination.



Tester Rigoureusement le Code

Valider le bon fonctionnement du code de gestion des données en effectuant des tests unitaires et d'intégration.

Évaluer la réactivité de l'application sous différentes charges de données en intégrant des tests de performance pour optimiser les performances.



Navigation entre écrans

A thick, hand-drawn style orange line underlining the text.

Naviguer entre deux écrans

Ce que vous allez apprendre dans ce chapitre :

- La navigation entre deux écrans : `MaterialPageRoute`, méthodes `push()` et `pop()`
- La création d'une nouvelle route



Naviguer avec Navigator



```
1  onPressed: () {  
2    Navigator.push(  
3      context,  
4      MaterialPageRoute(builder: (context) => SecondRoute()),  
5    );  
6  }  
7
```



```
1  onPressed: () {  
2    Navigator.pop(context);  
3  }
```

Utilisation de MaterialPageRoute dans Flutter



```
1 Navigator.push(  
2   context,  
3   MaterialPageRoute(builder: (context) => SecondScreen()),  
4 );
```



```
1 Navigator.pop(context, result);
```

Naviguer entre deux écrans

Navigation entre deux écrans : MaterialPageRoute, méthodes push() et pop()

Navigation entre deux écrans

```
onPressed: () {  
  Navigator.of(context).push(  
    MaterialPageRoute(  
      builder: (context) => const SongScreen(song: song),  
    ),  
  );  
},  
child: Text(song.name),
```

Parce que **Navigator** conserve une pile de **Route** objet (représentant la pile d'historique), la **push()** méthode prend également un objet **Route**.



Remarque

- MaterialPageRoute objet est une sous-classe de Route qui spécifie les animations de transition pour Material Design.

()

 $\{ \}$ 

Définir des pages à l'aide de MaterialApp



```
1  MaterialApp(  
2    home: HomePage(),  
3    routes: {  
4      '/page1': (context) => Page1(),  
5      '/page2': (context) => Page2(),  
6    },  
7  )
```

Naviguez à l'aide du Navigator



```
1 Navigator.pushNamed(context, '/page1');  
2  
3 Navigator.pushReplacementNamed(context, '/page2');
```

Accédez aux pages contenant des données



```
1 Navigator.pushNamed(context, '/page1', arguments: data);
```



```
1 class Page1 extends StatelessWidget {  
2   @override  
3   Widget build(BuildContext context) {  
4     final data = ModalRoute.of(context).settings.arguments;  
5     return Scaffold(  
6       // ...  
7     );  
8   }  
9 }
```

Surveillance de la navigation par onGenerateRoute



```
1  MaterialApp(  
2    onGenerateRoute: (settings) {  
3      if (settings.name == '/second') {  
4        return MaterialPageRoute(builder: (context) => SecondScreen());  
5      }  
6    },  
7  );
```

Activité 2

Création facile d'une nouvelle route

Créer deux écrans avec retour en arrière

Sous Android, une route équivaut à une activité. Dans iOS à un ViewController et dans Flutter, une route n'est qu'un **widget**.

```
class FirstRoute extends StatelessWidget {  
  const FirstRoute({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      appBar: AppBar(  
        title: const Text('First Route'), ),  
      body: Center(  
        child: ElevatedButton(  
          child: const Text('Open route'),  
          onPressed: () { // Navigate to second route when tapped.},  
        ),  
      ),  
    );  
  }  
}
```



Remarques

- Dans Flutter, les écrans et les pages sont appelés routes .

First Route

Open route

La 1ère route

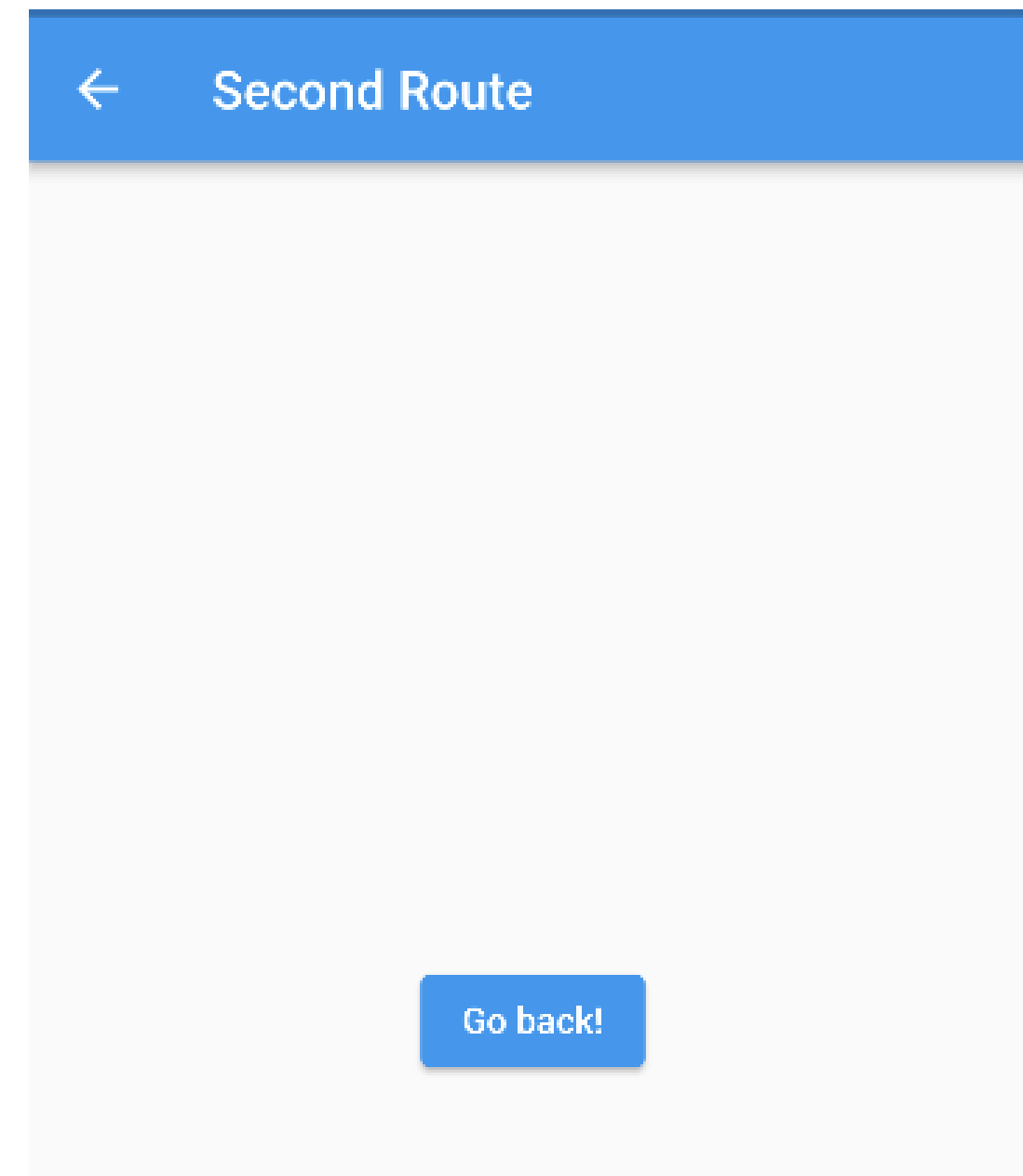
Activité 1

Création facile d'une nouvelle route

Créer deux écrans avec retour en arrière

Le 2ème écran avec la possibilité du retour en arrière.

```
class SecondRoute extends StatelessWidget {  
  const SecondRoute({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      appBar: AppBar(  
        title: const Text('Second Route'),  
      ),  
      body: Center(  
        child: ElevatedButton(  
          onPressed: () {  
            // Navigate back to first route when tapped.  
          },  
          child: const Text('Go back!'),  
        ),  
      ),  
    );  
  }  
}
```



La 2ème route

Activité 2

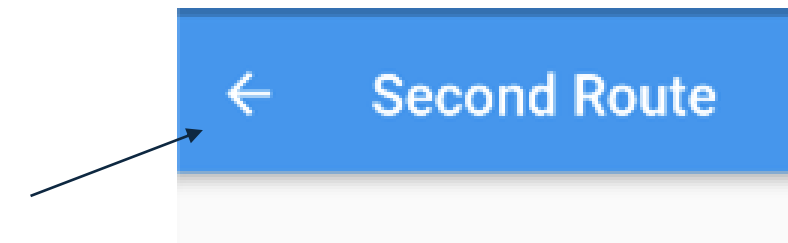
Création facile d'une nouvelle route

Accédez à la deuxième route à l'aide de Navigator.push()

```
// Within the `FirstRoute` widget  
onPressed: () {  
  Navigator.push(  
    context,  
    MaterialPageRoute(builder: (context) => const SecondRoute()),  
  );  
}
```

Revenez au premier écran en utilisant Navigator.pop()

```
// Within the SecondRoute widget  
onPressed: () {  
  Navigator.pop(context);  
}
```



La flèche de retour en arrière

Application ChatBot utilisant l'API ChatGPT dans Flutter

Projet

Le travail demandé pour couronner les activités de cet élément de module est de réaliser une application ChatBot qui interagit avec ChatGPT. Les différentes étapes pour réaliser ce projet dans sa version minimaliste sont données dans le blog suivant: <https://adwoadadson60.medium.com/in-this-article-we-will-discuss-how-to-build-a-simple-chatgpt-mobile-application-using-flutter-and-20ad7c421fc6>.

Il reste qu'une partie de l'évaluation prendra en compte les améliorations apportées au projet (design, comportement, navigation, authentification et animation).

Build a ChatGPT Mobile Application using Flutter and OpenAI



+



=

